

---

# **Panflute Documentation**

***Release 2.1.4***

**Sergio Correia**

**Jan 25, 2022**



# CONTENTS

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Motivation</b>                                     | <b>3</b>  |
| 1.1      | 1. Pythonic . . . . .                                 | 3         |
| 1.2      | 2. Detects common mistakes . . . . .                  | 3         |
| 1.3      | 3. Comes with batteries included . . . . .            | 3         |
| <b>2</b> | <b>Examples of panflute filters</b>                   | <b>5</b>  |
| 2.1      | Alternative: filters based on pandocfilters . . . . . | 5         |
| <b>3</b> | <b>Contents:</b>                                      | <b>7</b>  |
| 3.1      | User guide . . . . .                                  | 7         |
| 3.1.1    | A Simple filter . . . . .                             | 7         |
| 3.1.2    | More complex filters . . . . .                        | 7         |
| 3.1.3    | Globals and backmatter . . . . .                      | 8         |
| 3.1.4    | Using the included batteries . . . . .                | 10        |
| 3.1.5    | YAML code blocks . . . . .                            | 11        |
| 3.1.6    | Calling external programs . . . . .                   | 12        |
| 3.1.7    | Navigating through the document tree . . . . .        | 13        |
| 3.1.8    | Running filters automatically . . . . .               | 14        |
| 3.2      | Installation . . . . .                                | 15        |
| 3.2.1    | Dev Install . . . . .                                 | 15        |
| 3.2.2    | Source Code . . . . .                                 | 16        |
| 3.3      | Panflute API . . . . .                                | 16        |
| 3.3.1    | Base elements . . . . .                               | 16        |
| 3.3.2    | Standard elements . . . . .                           | 19        |
| 3.3.3    | Table-specific elements . . . . .                     | 27        |
| 3.3.4    | Standard functions . . . . .                          | 29        |
| 3.3.5    | “Batteries included” functions . . . . .              | 32        |
| 3.4      | Contributing . . . . .                                | 35        |
| 3.4.1    | License . . . . .                                     | 35        |
| <b>4</b> | <b>Indices and tables</b>                             | <b>37</b> |
|          | <b>Python Module Index</b>                            | <b>39</b> |
|          | <b>Index</b>                                          | <b>41</b> |



Panflute is a Python package that makes [Pandoc](#) filters fun to write. ([Installation](#))

*(Documentation last updated for panflute |version|)*

It is a pythonic alternative to John MacFarlane's [pandocfilters](#), from which it is heavily inspired.

To use it, write a function that works on Pandoc elements and call it through `run_filter`:

```
from panflute import *

def increase_header_level(elem, doc):
    if type(elem) == Header:
        if elem.level < 6:
            elem.level += 1
        else:
            return [] # Delete headers already in level 6

def main(doc=None):
    return run_filter(increase_header_level, doc=doc)

if __name__ == "__main__":
    main()
```



## MOTIVATION

Our goal is to make writing pandoc filters *as simple and clear as possible*. Starting from pandocfilters, we make it pythonic, add error and type checking, and include batteries for common tasks. In more detail:

### 1.1 1. Pythonic

- Elements are easier to modify. For instance, to change the level of a header, you can do `header.level += 1` instead of `header['c'][0] += 1`. To change the identifier, do `header.identifier = 'spam'` instead of `header['c'][1][1] = 'spam'`
- Elements are easier to create. Thus, to create a header you can do `Header(Str(The), Space, Str(Title), level=1, identifier=foo)` instead of `Header([1, ["foo", [], []], [{"t": "Str", "c": "The"}, {"t": "Space", "c": []}, {"t": "Str", "c": "Title"}])`
- You can navigate across elements. Thus, you can check if `isinstance(elem.parent, Inline)` or if `type(elem.next) == Space`

### 1.2 2. Detects common mistakes

- Check that the elements contain the correct types. Trying to create `Para('text')` will give you the error “Para() element must contain Inlines but received a str()”, instead of just failing silently when running the filter.

### 1.3 3. Comes with batteries included

- Convert markdown and other formatted strings into python objects or other formats, with the `convert_text(text, input_format, output_format)` function (which calls Pandoc internally)
- Use code blocks to hold YAML options and other data (such as CSV) with `yaml_filter(element, doc, tag, function)`.
- Called external programs to fetch results with `shell()`.
- Modifying the entire document (e.g. moving all the figures and tables to the back of a PDF) is easy, thanks to the `prepare` and `finalize` options of `run_filter`, and to the `replace_keyword` function
- Convenience elements such as `TableRow` and `TableCell` allow for easier filters.
- Panflute can be run as a filter itself, in which case it will run all filters listed in the metadata field `panflute-filters`.
- Can use metadata as a dict of builtin-values instead of Panflute objects, with `doc.get_metadata()`.





## EXAMPLES OF PANFLUTE FILTERS

Ports of existing pandocfilter modules are in the [github repo](#); additional and more advanced examples are in a [separate repository](#).

Also, a comprehensive list of filters and other Pandoc extras should be available [here](#) in the future.

### 2.1 Alternative: filters based on pandocfilters

- For a guide to pandocfilters, see the [repository](#) and the [tutorial](#).
- The repo includes [sample filters](#).
- The wiki lists useful [third party filters](#).



## CONTENTS:

### 3.1 User guide

#### 3.1.1 A Simple filter

Suppose we want to create a filter that sets all headers to level 1. For this, write this python script:

```
"""
Set all headers to level 1
"""

from panflute import *

def action(elem, doc):
    if isinstance(elem, Header):
        elem.level = 1

def main(doc=None):
    return run_filter(action, doc=doc)

if __name__ == '__main__':
    main()
```

---

**Note:** a more complete template is [located here](#)

---

#### 3.1.2 More complex filters

We might want filters that replace an element instead of just modifying it. For instance, suppose we want to replace all emphasized text with striked out text:

```
"""
Replace Emph elements with Strikeout elements
"""

from panflute import *

def action(elem, doc):
```

(continues on next page)

(continued from previous page)

```
    if isinstance(elem, Emph):
        return Strikeout(*elem.content)

def main(doc=None):
    return run_filter(action, doc=doc)

if __name__ == '__main__':
    main()
```

Or if we want to remove all tables:

```
"""
Remove all tables
"""

from panflute import *

def action(elem, doc):
    if isinstance(elem, Table):
        return []

def main(doc=None):
    return run_filter(action, doc=doc)

if __name__ == '__main__':
    main()
```

### 3.1.3 Globals and backmatter

Suppose we want to add a table of contents based on all headers, or move all tables to a specific location in the document. This requires tracking global variables (which can be stored as attributes of `doc`).

To add a table of contents at the beginning:

```
"""
Add table of contents at the beginning;
uses optional metadata value 'toc-depth'
"""

from panflute import *

def prepare(doc):
    doc.toc = BulletList()
    doc.depth = int(doc.get_metadata('toc-depth', default=1))
```

(continues on next page)

(continued from previous page)

```

def action(elem, doc):
    if isinstance(elem, Header) and elem.level <= doc.depth:
        item = ListItem(Plain(*elem.content))
        doc.toc.content.append(item)

def finalize(doc):
    doc.content.insert(0, doc.toc)
    del doc.toc, doc.depth

def main(doc=None):
    return run_filter(action, prepare=prepare, finalize=finalize, doc=doc)

if __name__ == '__main__':
    main()

```

To move all tables to the place where the string *\$tables* is:

```

"""
Move tables to where the string $tables is.
"""

from panflute import *

def prepare(doc):
    doc.backmatter = []

def action(elem, doc):
    if isinstance(elem, Table):
        doc.backmatter.append(elem)
    return []

def finalize(doc):
    div = Div(*doc.backmatter)
    doc = doc.replace_keyword('$tables', div)

def main(doc=None):
    return run_filter(action, prepare, finalize, doc=doc)

if __name__ == '__main__':
    main()

```

### 3.1.4 Using the included batteries

There are several functions and methods that make your life easier, such as the `replace_keyword` method shown above.

Other useful functions include `convert_text` (to load and parse markdown or other formatted text) and `stringify` (to extract the underlying text from an element and its children). For metadata, you can use the `doc.get_metadata` attribute to extract user-specified options (booleans, strings, etc.)

For instance, you can combine these functions to allow for include directives (so you can include and parse markdown files from other files).

```
"""
Panflute filter to allow file includes

Each include statement has its own line and has the syntax:

    $include ../somefolder/somefile

Each include statement must be in its own paragraph. That is, in its own line
and separated by blank lines.

If no extension was given, ".md" is assumed.
"""

import os
import panflute as pf

def is_include_line(elem):
    if len(elem.content) < 3:
        return False
    elif not all (isinstance(x, (pf.Str, pf.Space)) for x in elem.content):
        return False
    elif elem.content[0].text != '$include':
        return False
    elif type(elem.content[1]) != pf.Space:
        return False
    else:
        return True

def get_filename(elem):
    fn = pf.stringify(elem, newlines=False).split(maxsplit=1)[1]
    if not os.path.splitext(fn)[1]:
        fn += '.md'
    return fn

def action(elem, doc):
    if isinstance(elem, pf.Para) and is_include_line(elem):

        fn = get_filename(elem)
        if not os.path.isfile(fn):
            return
```

(continues on next page)

(continued from previous page)

```

    with open(fn) as f:
        raw = f.read()

    new_elems = pf.convert_text(raw)

    # Alternative A:
    return new_elems
    # Alternative B:
    # div = pf.Div(*new_elems, attributes={'source': fn})
    # return div

def main(doc=None):
    return pf.run_filter(action, doc=doc)

if __name__ == '__main__':
    main()

```

### 3.1.5 YAML code blocks

A YAML filter is a filter that parses fenced code blocks that contain YAML metadata. For instance:

```

Some text

~~~ csv
title: Some Title
has-header: True
---
Col1, Col2, Col3
1, 2, 3
10, 20, 30
~~~

More text

```

Note that fenced code blocks use three or more [tildes](#) or [backticks](#) as separators. Within a code block, use [three hyphens](#) or [three dots](#) to separate the YAML options from the rest of the block.

As an example, we will design a filter that will be applied to all code blocks with the `csv` class, like the one shown above. To avoid boilerplate code (such as parsing the YAML part), we use the useful [yaml\\_filter](#) function:

```

"""
Panflute filter to parse CSV in fenced YAML code blocks
"""

import io
import csv
import panflute as pf

```

(continues on next page)

(continued from previous page)

```
def fenced_action(options, data, element, doc):
    # We'll only run this for CodeBlock elements of class 'csv'
    title = options.get('title', 'Untitled Table')
    title = [pf.Str(title)]
    has_header = options.get('has-header', False)

    with io.StringIO(data) as f:
        reader = csv.reader(f)
        body = []
        for row in reader:
            cells = [pf.TableCell(pf.Plain(pf.Str(x))) for x in row]
            body.append(pf.TableRow(*cells))

    header = body.pop(0) if has_header else None
    table = pf.Table(*body, header=header, caption=title)
    return table

def main(doc=None):
    return pf.run_filter(pf.yaml_filter, tag='csv', function=fenced_action,
                        doc=doc)

if __name__ == '__main__':
    main()
```

---

**Note:** a more complete template is [here](#) , a fully developed filter for CSVs is [also available](#).

---

---

**Note:** `yaml_filter` now allows a `strict_yaml=True` option, which allows multiple YAML blocks, but with the caveat that all YAML blocks must start with `—` and end with `—` or `...`.

---

### 3.1.6 Calling external programs

We might also want to embed results from other programs.

One option is to do so through Python’s internals. For instance, we can use fetch data from wikipedia and show it on the document. Thus, the following script will replace links like these: `[Pandoc](wiki://)` With this “Pandoc is a free and open-source software document converter...”.

```
"""
Panflute filter that embeds wikipedia text

Replaces markdown such as [Stack Overflow](wiki://) with the resulting text.
"""

import requests
import panflute as pf
```

(continues on next page)



(continued from previous page)

```

def action(elem, doc):
    if isinstance(elem, pf.Link) and elem.url.startswith('wiki://'):
        title = pf.stringify(elem).strip()
        baseurl = 'https://en.wikipedia.org/w/api.php'
        query = {'format': 'json', 'action': 'query', 'prop': 'extracts',
                  'explaintext': '', 'titles': title}
        r = requests.get(baseurl, params=query)
        data = r.json()
        extract = list(data['query']['pages'].values())[0]['extract']
        extract = extract.split('.', maxsplit=1)[0]
        return pf.RawInline(extract)

def main(doc=None):
    return pf.run_filter(action, doc=doc)

if __name__ == '__main__':
    main()

```

Alternatively, we might want to run other programs through the shell. For this, explore the [shell](#) function.

### 3.1.7 Navigating through the document tree

You might wish to apply a filter that depends on the parent or sibling objects of an element. For instance, Modify the first row (*TableRow*) of a table, or all the *Str* items nested within a header.

For this, every element has a *.parent* attribute (and the related *.next*, *.prev*, *.ancestor(#)*, *.index*, *.offset(#)* attributes).

For example, the code below will emphasize all text in the last row of every table:

```

"""
Make text in the last row of every table bold
"""

import panflute as pf

def action(elem, doc):
    if isinstance(elem, pf.TableRow):
        # Exclude table headers (which are not in a list)
        if elem.index is None:
            return

        if elem.next is None:
            pf.debug(elem)
            elem.walk(make_emph)

def make_emph(elem, doc):

```

(continues on next page)

(continued from previous page)

```
if isinstance(elem, pf.Str):
    return pf.Emph(elem)

def main(doc=None):
    return pf.run_filter(action, doc=doc)

if __name__ == '__main__':
    main()
```

### 3.1.8 Running filters automatically

If you run panflute as a filter (`pandoc ... -F panflute`), then panflute will run all filters specified in the metadata field `panflute-filters`. This is faster and more convenient than typing the precise list and order of filters used every time the document is run.

You can also specify the location of the filters with the `panflute-path` field, which will take precedence over Pandoc's search [locations](#) (`..`, `$datadir/filters`, and `$path`).

Example:

```
---
title: Some title
panflute-filters: [remove-tables, include]
panflute-path: 'panflute/docs/source'
...

Lorem ipsum
```

For this to work, the filters need to have a very specific structure, with a `main()` function of the following form:

```
"""
Pandoc filter using panflute
"""

import panflute as pf

def prepare(doc):
    pass

def action(elem, doc):
    if isinstance(elem, pf.Element) and doc.format == 'latex':
        pass
        # return None -> element unchanged
        # return [] -> delete element

def finalize(doc):
    pass
```

(continues on next page)

(continued from previous page)

```
def main(doc=None):
    return pf.run_filter(action,
                        prepare=prepare,
                        finalize=finalize,
                        doc=doc)

if __name__ == '__main__':
    main()
```

**Note:** To be able to run filters automatically, the main function needs to be exactly as shown, with an optional argument `doc`, that gets passed to `run_filter`, and which is returned back.

**Note:** You can add `panflute-verbose: true` to the metadata to display debugging information, including the folders searched and the filters executed.

## 3.2 Installation

To install panflute from PyPI, open the command line and type:

```
pip install panflute
```

- Works with Python 3.6+, and PyPy
- On Windows, you might need to open the command line (cmd) as administrator (*ctrl+shift+enter*).

To install the latest Github version of panflute, type:

```
pip install git+git://github.com/sergiocorreia/panflute.git
```

### 3.2.1 Dev Install

After cloning the Github repo into your computer, you can install the package locally:

```
python setup.py install
```

Alternatively, you can install it through a symlink, so changes are automatically updated:

```
python setup.py develop
```

### 3.2.2 Source Code

To browse the source code, report issues or contribute, check the [github repository](#).

## 3.3 Panflute API

### Contents:

- *Base elements*
  - *Low-level classes*
- *Standard elements*
- *Table-specific elements*
- *Standard functions*
- *“Batteries included” functions*

### 3.3.1 Base elements

**class** `Element(*args, **kwargs)`

Base class of all Pandoc elements

**parent**

Element that contains the current one.

**Note:** the `.parent` and related attributes are not implemented for metadata elements.

**Return type** `Element` | `None`

**location**

`None` unless the element is in a non-standard location of its parent, such as the `.caption` or `.header` attributes of a table.

In those cases, `.location` will be equal to a string.

**rtype** `str` | `None`

**walk**(`action`, `doc=None`, `walk_inlines=True`)

Walk through the element and all its children (sub-elements), applying the provided function `action`.

A trivial example would be:

```
from panflute import *

def no_action(elem, doc):
    pass

doc = Doc(Para(Str('a')))
altered = doc.walk(no_action)
```

**Parameters**

- **action** (function) – function that takes (element, doc) as arguments.

- **doc** (Doc) – root document; used to access metadata, the output format (in `.format`, other elements, and other variables). Only use this variable if for some reason you don't want to use the current document of an element.

**Return type** `Element` | `[]` | `None`

#### **content**

Sequence of `Element` objects (usually either `Block` or `Inline`) that are “children” of the current element.

Only available for elements that accept `*args`.

Note: some elements have children in attributes other than `content` (such as `Table` that has children in the `header` and `caption` attributes).

#### **index**

Return position of element inside the parent.

**Return type** `int` | `None`

#### **ancestor(*n*)**

Return the *n*-th ancestor. Note that `elem.ancestor(1) == elem.parent`

**Return type** `Element` | `None`

#### **offset(*n*)**

Return a sibling element offset by *n*

**Return type** `Element` | `None`

#### **prev**

Return the previous sibling. Note that `elem.offset(-1) == elem.prev`

**Return type** `Element` | `None`

#### **next**

Return the next sibling. Note that `elem.offset(1) == elem.next`

**Return type** `Element` | `None`

#### **replace\_keyword(*keyword*, *replacement* [, *count* ])**

Walk through the element and its children and look for `Str()` objects that contains exactly the `keyword`. Then, replace it.

Usually applied to an entire document (a `Doc` element)

Note: If the replacement is a block, it cannot be put in place of a `Str` element. As a solution, the closest ancestor (e.g. the parent) will be replaced instead, but only if possible (if the parent only has one child).

Example:

```
>>> from panflute import *
>>> p1 = Para(Str('Spam'), Space, Emph(Str('and')), Space, Str('eggs'))
>>> p2 = Para(Str('eggs'))
>>> p3 = Plain(Emph(Str('eggs')))
>>> doc = Doc(p1, p2, p3)
>>> doc.content
ListContainer(Para(Str(Spam) Space Emph(Str(and) Space Str(eggs)))
↳ Para(Str(eggs)) Plain(Emph(Str(eggs))))
>>> doc.replace_keyword('eggs', Str('ham'))
>>> doc.content
ListContainer(Para(Str(Spam) Space Emph(Str(and) Space Str(ham)))
↳ Para(Str(ham)) Plain(Emph(Str(ham))))
```

(continues on next page)

(continued from previous page)

```
>>> doc.replace_keyword(keyword='ham', replacement=Para(Str('spam')))  
>>> doc.content  
ListContainer(Para(Str(Spam) Space Emph(Str(and) Space Str(ham)))  
↳ Para(Str(spam)) Para(Str(spam)))
```

### Parameters

- **keyword** (`str`) – string that will be searched (cannot have spaces!)
- **replacement** (`Element`) – element that will be placed in turn of the `Str` element that contains the keyword.
- **count** (`int`) – number of occurrences that will be replaced. If count is not given or is set to zero, all occurrences will be replaced.

### container

Rarely used attribute that returns the `ListContainer` or `DictContainer` that contains the element (or returns `None` if no such container exist)

**Return type** `ListContainer | DictContainer | None`

---

The following elements inherit from `Element`:

Base classes and methods of all Pandoc elements

**class** `Block(*args, **kwargs)`

Base class of all block elements

**class** `Inline(*args, **kwargs)`

Base class of all inline elements

**class** `MetaValue(*args, **kwargs)`

Base class of all metadata elements

## Low-level classes

*(Skip unless you want to understand the internals)*

These containers keep track of the identity of the parent object, and the attribute of the parent object that they correspond to.

**class** `DictContainer(*args, oktypes=<class 'object'>, parent=None, **kwargs)`

Wrapper around a dict, to track the elements' parents. **This class shouldn't be instantiated directly by users, but by the elements that contain it.**

### Parameters

- **args** – elements contained in the dict-like object
- **oktypes** (`type | tuple`) – type or tuple of types that are allowed as items
- **parent** (`Element`) – the parent element

**class** `ListContainer(*args, oktypes=<class 'object'>, parent=None)`

Wrapper around a list, to track the elements' parents. **This class shouldn't be instantiated directly by users, but by the elements that contain it.**

### Parameters

- **args** – elements contained in the list-like object
- **oktypes** (type | tuple) – type or tuple of types that are allowed as items
- **parent** (Element) – the parent element
- **container** (str | None) – None, unless the element is not part of its .parent.content (this is the case for table headers for instance, which are not retrieved with table.content but with table.header)

**insert**(i, v)  
S.insert(index, value) – insert value before index

**Note:** To keep track of every element’s parent we do some class magic. Namely, `Element.content` is not a list attribute but a property accessed via getter and setters. Why?

```
>>> e = Para(Str(Hello), Space, Str(World!))
```

This creates a `Para` element, which stores the three inline elements (`Str`, `Space` and `Str`) inside an `.content` attribute. If we add `.parent` attributes to these elements, there are three ways they can be made obsolete:

1. By replacing specific elements: `e.content[0] = Str('Bye')`
2. By replacing the entire list: `e.contents = other_items`

We deal with the first problem with wrapping the list of items with a `ListContainer` class of type `collections.MutableSequence`. This class updates the `.parent` attribute to elements returned through `__getitem__` calls.

For the second problem, we use setters and getters which update the `.parent` attribute.

### 3.3.2 Standard elements

These are the standard Pandoc elements, as described [here](#). Consult the [repo](#) for the latest updates.

**Note:** The attributes of every element object will be i) the parameters listed below, plus ii) the attributes of `Element`. Example:

```
>>> h = Str(text='something')
>>> h.text
'something'
>>> hasattr(h, 'parent')
True
```

**Exception:** the `.content` attribute only exists in elements that take `*args` (so we can do `Para().content` but not `Str().content`).

Classes corresponding to Pandoc elements

Notation: - “ica” is shorthand for “identifier, classes, attributes”

**class Doc**(\*args, \*\*kwargs)  
Pandoc document container.

Besides the document, it includes the frontpage metadata and the desired output format. Filter functions can also add properties to it as means of global variables that can later be read by different calls.

#### Parameters

- **args** (Block sequence) – top-level documents contained in the document
- **metadata** (dict) – the frontpage metadata
- **format** (str) – output format, such as ‘markdown’, ‘latex’ and ‘html’
- **api\_version** (tuple) – A tuple of three ints of the form (1, 18, 0)

**Returns** Document with base class Element

**Base** Element

**Example**

```
>>> meta = {'author': 'John Doe'}
>>> content = [Header(Str('Title')), Para(Str('Hello!'))]
>>> doc = Doc(*content, metadata=meta, format='pdf')
>>> doc.figure_count = 0 # You can add attributes freely
```

**get\_metadata**([*key*, *default*, *simple*])

Retrieve metadata with nested keys separated by dots.

This is useful to avoid repeatedly checking if a dict exists, as the frontmatter might not have the keys that we expect.

With *builtin=True* (the default), it will convert the results to built-in Python types, instead of *MetaValue* elements. EG: instead of returning a *MetaBool* it will return *True|False*.

**Parameters**

- **key** (str) – string with the keys separated by a dot (*key1.key2*). Default is an empty string (which returns the entire metadata dict)
- **default** – return value in case the key is not found (default is *None*)
- **builtin** – If *True*, return built-in Python types (default is *True*)

**Example**

```
>>> doc.metadata['format']['show-frame'] = True
>>> # ...
>>> # afterwards:
>>> show_frame = doc.get_metadata('format.show-frame', False)
>>> stata_path = doc.get_metadata('media.path.figures', '.')
```

---

Classes corresponding to Pandoc elements

Notation: - “ica” is shorthand for “identifier, classes, attributes”

**class BlockQuote**(\*args, \*\*kwargs)

Block quote

**Parameters** **args** (Block) – sequence of blocks

**Base** Block

**class BulletList**(\*args, \*\*kwargs)

Bullet list (unordered list)

**Parameters** **args** (*ListItem* | list) – List item

**Base** Block



**class Citation(\*args, \*\*kwargs)**  
 A single citation to a single work

**Parameters**

- **id** (**str**) – citation key (e.g. the BibTeX keyword)
- **mode** (**str**) – how will the citation appear ('NormalCitation' for the default style, 'AuthorIn-Text' to exclude parenthesis, 'SuppressAuthor' to exclude the author's name)
- **prefix** ([**Inline**]) – Text before the citation reference
- **suffix** ([**Inline**]) – Text after the citation reference
- **note\_num** (**int**) – (Not sure...)
- **hash** (**int**) – (Not sure...)

**Base Element**

**class Cite(\*args, \*\*kwargs)**  
 Cite: set of citations with related text

**Parameters**

- **args** (**Inline**) – contents of the cite (the raw text)
- **citations** ([**Citation**]) – sequence of citations

**Base Inline**

**class Code(\*args, \*\*kwargs)**  
 Inline code (literal)

**Parameters**

- **text** (**str**) – literal text (preformatted text, code, etc.)
- **identifier** (**str**) – element identifier (usually unique)
- **classes** (**list** of **str**) – class names of the element
- **attributes** (**dict**) – additional attributes

**Base Inline**

**class CodeBlock(\*args, \*\*kwargs)**  
 Code block (literal text) with optional attributes

**Parameters**

- **text** (**str**) – literal text (preformatted text, code, etc.)
- **identifier** (**str**) – element identifier (usually unique)
- **classes** (**list** of **str**) – class names of the element
- **attributes** (**dict**) – additional attributes

**Base Block**

**class Definition(\*args, \*\*kwargs)**  
 The definition (description); used in a definition list. It can include code and all other block elements.

**Parameters** **args** (**Block**) – elements

**Base Element**

**class DefinitionItem(\*args, \*\*kwargs)**

Contains pairs of Term and Definitions (plural!)

Each list item represents a pair of i) a term (a list of inlines) and ii) one or more definitions

**Parameters**

- **term** ([Inline]) – Term of the definition (an inline holder)
- **definitions** – List of definitions or descriptions (each a block holder)

**Base Element**

**class DefinitionList(\*args, \*\*kwargs)**

Definition list: list of definition items; basically (term, definition) tuples.

Each list item represents a pair of i) a term (a list of inlines) and ii) one or more definitions (each a list of blocks)

Example:

```
>>> term1 = [Str('Spam')]
>>> def1 = Definition(Para(Str('...emails')))
>>> def2 = Definition(Para(Str('...meat')))
>>> spam = DefinitionItem(term1, [def1, def2])
>>>
>>> term2 = [Str('Spanish'), Space, Str('Inquisition')]
>>> def3 = Definition(Para(Str('church'), Space, Str('court')))
>>> inquisition = DefinitionItem(term=term2, definitions=[def3])
>>> definition_list = DefinitionList(spam, inquisition)
```

**Parameters** **args** (*DefinitionItem*) – Definition items (a term with definitions)

**Base Block**

**class Div(\*args, \*\*kwargs)**

Generic block container with attributes

**Parameters**

- **args** (Block) – contents of the div
- **identifier** (*str*) – element identifier (usually unique)
- **classes** (*list* of *str*) – class names of the element
- **attributes** (*dict*) – additional attributes

**Base Block**

**class Emph(\*args, \*\*kwargs)**

Emphasized text

**Parameters** **args** (Inline) – elements that will be emphasized

**Base Inline**

**class Header(\*args, \*\*kwargs)**

**Parameters**

- **args** (Inline) – contents of the header
- **level** (int) – level of the header (1 is the largest and 6 the smallest)
- **identifier** (*str*) – element identifier (usually unique)

- **classes** (list of str) – class names of the element
- **attributes** (dict) – additional attributes

Base Block

Example

```
>>> title = [Str('Monty'), Space, Str('Python')]
>>> header = Header(*title, level=2, identifier='toc')
>>> header.level += 1
```

**class HorizontalRule(\*args, \*\*kwargs)**

Horizontal rule

Base Block

**class Image(\*args, \*\*kwargs)**

Parameters

- **args** (Inline) – text with the image description
- **url** (str) – URL or path of the image
- **title** (str) – Alt. title
- **identifier** (str) – element identifier (usually unique)
- **classes** (list of str) – class names of the element
- **attributes** (dict) – additional attributes

Base Inline

**class LineBlock(\*args, \*\*kwargs)**

Line block (sequence of lines)

Parameters **args** (*LineItem* | list) – Line item

Base Block

**class LineBreak(\*args, \*\*kwargs)**

Hard line break

Base Inline

**class LineItem(\*args, \*\*kwargs)**

Line item (contained in line blocks)

Parameters **args** (Inline) – Line item

Base Element

**class Link(\*args, \*\*kwargs)**

Hyperlink

Parameters

- **args** (Inline) – text with the link description
- **url** (str) – URL or path of the link
- **title** (str) – Alt. title
- **identifier** (str) – element identifier (usually unique)

- **classes** (list of str) – class names of the element
- **attributes** (dict) – additional attributes

Base Inline

**class ListItem**(\*args, \*\*kwargs)

List item (contained in bullet lists and ordered lists)

**Parameters** **args** (Block) – List item

Base Element

**class Math**(\*args, \*\*kwargs)

TeX math (literal)

**Parameters**

- **text** (str) – a string of raw text representing TeX math
- **format** (str) – How the math will be typeset ('DisplayMath' or 'InlineMath')

Base Inline

**class MetaBlocks**(\*args, \*\*kwargs)

MetaBlocks: list of arbitrary blocks within the metadata

**Parameters** **args** (Block) – sequence of block elements

Base MetaValue

**class MetaBool**(\*args, \*\*kwargs)

Container for True/False metadata values

**Parameters** **boolean** (bool) – True/False value

Base MetaValue

**class MetaInlines**(\*args, \*\*kwargs)

MetaInlines: list of arbitrary inlines within the metadata

**Parameters** **args** (Inline) – sequence of inline elements

Base MetaValue

**class MetaList**(\*args, \*\*kwargs)

Metadata list container

**Parameters** **args** (MetaValue) – contents of a metadata list

Base MetaValue

**class MetaMap**(\*args, \*\*kwargs)

Metadata container for ordered dicts

**Parameters**

- **args** (MetaValue) – (key, value) tuples
- **kwargs** (MetaValue) – named arguments

Base MetaValue

**property content**

Map of MetaValue objects.

**class MetaString**(\*args, \*\*kwargs)

Text (a string)

**Parameters** **text** (`str`) – a string of unformatted text

**Base** `MetaValue`

**class** `Note(*args, **kwargs)`

Footnote or endnote

**Parameters** **args** (`Block`) – elements that are part of the note

**Base** `Inline`

**class** `Null(*args, **kwargs)`

Nothing

**Base** `Block`

**class** `OrderedList(*args, **kwargs)`

Ordered list (attributes and a list of items, each a list of blocks)

**Parameters**

- **args** (`ListItem` | `list`) – List item
- **start** (`int`) – Starting value of the list
- **style** (`str`) – Style of the number delimiter ('DefaultStyle', 'Example', 'Decimal', 'Lower-Roman', 'UpperRoman', 'LowerAlpha', 'UpperAlpha')
- **delimiter** (`str`) – List number delimiter ('DefaultDelim', 'Period', 'OneParen', 'Two-Parens')

**Base** `Block`

**class** `Para(*args, **kwargs)`

Paragraph

**Parameters** **args** (`Inline`) – contents of the paragraph

**Base** `Block`

**Example**

```
>>> content = [Str('Some'), Space, Emph(Str('words.'))]
>>> para1 = Para(*content)
>>> para2 = Para(Str('More'), Space, Str('words.'))
```

**class** `Plain(*args, **kwargs)`

Plain text, not a paragraph

**Parameters** **args** (`Inline`) – contents of the plain block of text

**Base** `Block`

**class** `Quoted(*args, **kwargs)`

Quoted text

**Parameters**

- **args** (`Inline`) – contents of the quote
- **quote\_type** (`str`) – either 'SingleQuote' or 'DoubleQuote'

**Base** `Inline`

**class** `RawBlock(*args, **kwargs)`

Raw block

**Parameters**

- **text** (`str`) – a string of raw text with another underlying format
- **format** (`str`) – Format of the raw text ('html', 'tex', 'latex', 'context', etc.)

**Base Block****class RawInline**(\*args, \*\*kwargs)

Raw inline text

**Parameters**

- **text** (`str`) – a string of raw text with another underlying format
- **format** (`str`) – Format of the raw text ('html', 'tex', 'latex', 'context', etc.)

**Base Inline****class SmallCaps**(\*args, \*\*kwargs)

Small caps text (list of inlines)

**Parameters** **args** (`Inline`) – elements that will be set with small caps**Base Inline****class SoftBreak**(\*args, \*\*kwargs)

Soft line break

**Base Inline****class Space**(\*args, \*\*kwargs)

Inter-word space

**Base Inline****class Span**(\*args, \*\*kwargs)

Generic block container with attributes

**Parameters**

- **args** (`Inline`) – contents of the div
- **identifier** (`str`) – element identifier (usually unique)
- **classes** (`list` of `str`) – class names of the element
- **attributes** (`dict`) – additional attributes

**Base Inline****class Str**(\*args, \*\*kwargs)

Text (a string)

**Parameters** **text** (`str`) – a string of unformatted text**Base Inline****class Strikeout**(\*args, \*\*kwargs)

Strikeout text

**Parameters** **args** (`Inline`) – elements that will be stricken out**Base Inline****class Strong**(\*args, \*\*kwargs)

Strongly emphasized text

**Parameters** **args** (`Inline`) – elements that will be emphasized

**Base Inline**

**class Subscript**(\*args, \*\*kwargs)  
Subscripted text (list of inlines)

**Parameters** **args** (Inline) – elements that will be set subscript

**Base Inline**

**class Superscript**(\*args, \*\*kwargs)  
Superscripted text (list of inlines)

**Parameters** **args** (Inline) – elements that will be set superscript

**Base Inline**

**class Underline**(\*args, \*\*kwargs)  
Underlined text

**Parameters** **args** (Inline) – elements that will be underlined

**Base Inline**

### 3.3.3 Table-specific elements

Classes corresponding to Pandoc Table elements

**class Caption**(\*args, \*\*kwargs)  
Table caption with optional short caption

**Parameters**

- **args** (Block) – caption
- **short\_caption** (list of Inline) – Short caption
- **identifier** – element identifier (usually unique)

**Base Element**

**class Table**(\*args, \*\*kwargs)

Table, composed of a table head, one or more table bodies, and a a table foot. You can also specify captions, short captions, column alignments, and column widths.

Example:

```
>>> x = [Para(Str('Something')), Para(Space, Str('else'))]
>>> c1 = TableCell(*x)
>>> c2 = TableCell(Header(Str('Title')))
>>> row = TableRow(c1, c2)
>>>
>>> body = TableBody(row)
>>> head = TableHead(row)
>>> caption = Caption(Para(Str('Title')))
>>> table = Table(body, head=head, caption=caption)
```

TODO: UPDATE EXAMPLE TODO: OFFER A SIMPLE WAY TO BUILD A TABLE, with e.g. .alignments and .widths

**Parameters**

- **args** (*TableBody*) – Table bodies

- **head** (*TableHead*) – Table head
- **foot** (*TableFoot*) – Table foot
- **caption** (*Caption*) – The caption of the table (with optional short caption)
- **colspec** (*list* of (*Alignment*, *ColWidth*)) – list of (alignment, colwidth) tuples; one for each column
- **identifier** (*str*) – element identifier (usually unique)
- **classes** (*list* of *str*) – class names of the element
- **attributes** (*dict*) – additional attributes
- **alignment** (*[str]*) – List of row alignments (either ‘AlignLeft’, ‘AlignRight’, ‘AlignCenter’ or ‘AlignDefault’).
- **colwidth** (*[float | “ColWidthDefault”]*) – Fractional column widths

Base Block

```
class TableBody(*args, **kwargs)
```

Body of a table, containing a list of intermediate head rows, a list of table body rows, *row\_head\_columns*, plus optional attributes

Parameters

- **row** (*str*) – head rows
- **head** (*list* of *TableRow*) – Intermediate head (list of table rows)
- **row\_head\_columns** (*class:int*) – number of columns on the left that are considered column headers (default: 0)
- **identifier** (*str*) – element identifier (usually unique)
- **classes** (*list* of *str*) – class names of the element
- **attributes** (*dict*) – additional attributes

Base Block

```
class TableCell(*args, **kwargs)
```

Table Cell

Parameters

- **args** (*Block*) – elements
- **alignment** (*str*) – row alignment (either ‘AlignLeft’, ‘AlignRight’, ‘AlignCenter’ or ‘AlignDefault’).
- **rowspan** (*int*) – number of rows occupied by a cell (height of a cell)
- **colspan** (*int*) – number of columns occupied by a cell (width of a cell)
- **identifier** (*str*) – element identifier (usually unique)
- **classes** (*list* of *str*) – class names of the element
- **attributes** (*dict*) – additional attributes

Base Element

```
class TableFoot(*args, **kwargs)
```

The foot of a table, containing a one or more foot rows, plus optional attributes

Parameters



- **row** (*str*) – foot rows
- **identifier** (*str*) – element identifier (usually unique)
- **classes** (*list* of *str*) – class names of the element
- **attributes** (*dict*) – additional attributes

Base Block

**class TableHead**(\*args, \*\*kwargs)

The head of a table, containing a one or more head rows, plus optional attributes

Parameters

- **row** (*str*) – head rows
- **identifier** (*str*) – element identifier (usually unique)
- **classes** (*list* of *str*) – class names of the element
- **attributes** (*dict*) – additional attributes

Base Block

**class TableRow**(\*args, \*\*kwargs)

Table Row

Parameters

- **args** (*TableCell*) – cells
- **identifier** (*str*) – element identifier (usually unique)
- **classes** (*list* of *str*) – class names of the element
- **attributes** (*dict*) – additional attributes

Base Element

### 3.3.4 Standard functions

|                                                        |                                                                                                                                                                                                           |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>run_filters</i> (actions[, prepare, finalize, ...]) | Receive a Pandoc document from the input stream (default is stdin), walk through it applying the functions in <i>actions</i> to each element, and write it back to the output stream (default is stdout). |
| <i>run_filter</i> (action, *args, **kwargs)            | Wrapper for <i>run_filters()</i>                                                                                                                                                                          |
| <i>toJSONFilter</i> (*args, **kwargs)                  | Wrapper for <i>run_filter()</i> , which calls <i>run_filters()</i>                                                                                                                                        |
| <i>toJSONFilters</i> (*args, **kwargs)                 | Wrapper for <i>run_filters()</i>                                                                                                                                                                          |
| <i>load</i> ([input_stream])                           | Load JSON-encoded document and return a Doc element.                                                                                                                                                      |
| <i>dump</i> (doc[, output_stream])                     | Dump a Doc object into a JSON-encoded text string.                                                                                                                                                        |

See also:

The *walk()* function has been replaced by the *Element.walk()* method of each element. To walk through the entire document, do `altered = doc.walk()`.

I/O related functions

**dump**(*doc*, *output\_stream=None*)

Dump a Doc object into a JSON-encoded text string.

The output will be sent to `sys.stdout` unless an alternative text stream is given.

To dump to `sys.stdout` just do:

```
>>> import panflute as pf
>>> doc = pf.Doc(Para(Str('a'))) # Create sample document
>>> pf.dump(doc)
```

To dump to file:

```
>>> with open('some-document.json', 'w', encoding='utf-8') as f:
>>>     pf.dump(doc, f)
```

To dump to a string:

```
>>> import io
>>> with io.StringIO() as f:
>>>     pf.dump(doc, f)
>>> contents = f.getvalue()
```

#### Parameters

- **doc** (Doc) – document, usually created with `load()`
- **output\_stream** – text stream used as output (default is `sys.stdout`)

**load**(*input\_stream=None*)

Load JSON-encoded document and return a Doc element.

The JSON input will be read from `sys.stdin` unless an alternative text stream is given (a file handle).

To load from a file, you can do:

```
>>> import panflute as pf
>>> with open('some-document.json', encoding='utf-8') as f:
>>>     doc = pf.load(f)
```

To load from a string, you can do:

```
>>> import io
>>> raw = '[{"unMeta":{}},'
>>> [{"t":"Para","c":[{"t":"Str","c":"Hello!"}]}]'
>>> f = io.StringIO(raw)
>>> doc = pf.load(f)
```

**Parameters** **input\_stream** – text stream used as input (default is `sys.stdin`)

**Return type** Doc

**run\_filter**(*action*, *\*args*, *\*\*kwargs*)

Wrapper for `run_filters()`

Receive a Pandoc document from stdin, apply the *action* function to each element, and write it back to stdout.

See `run_filters()`

**run\_filters**(*actions*, *prepare=None*, *finalize=None*, *input\_stream=None*, *output\_stream=None*, *doc=None*, *walk\_inlines=True*, *\*\*kwargs*)

Receive a Pandoc document from the input stream (default is stdin), walk through it applying the functions in *actions* to each element, and write it back to the output stream (default is stdout).

Notes:

- It receives and writes the Pandoc documents as JSON-encoded strings; this is done through the [load\(\)](#) and [dump\(\)](#) functions.
- It walks through the document once for every function in *actions*, so the actions are applied sequentially.
- By default, it will read from stdin and write to stdout, but these can be modified.
- It can also apply functions to the entire document at the beginning and end; this allows for global operations on the document.
- If *doc* is a Doc instead of None, **run\_filters** will return the document instead of writing it to the output stream.

#### Parameters

- **actions** ([function]) – sequence of functions; each function takes (element, doc) as argument, so a valid header would be `def action(elem, doc):`
- **prepare** (function) – function executed at the beginning; right after the document is received and parsed
- **finalize** (function) – function executed at the end; right before the document is converted back to JSON and written to stdout.
- **input\_stream** – text stream used as input (default is `sys.stdin`)
- **output\_stream** – text stream used as output (default is `sys.stdout`)
- **doc** (None | Doc) – None unless running panflute as a filter, in which case this will be a Doc element
- **\*kwargs** – keyword arguments will be passed through to the *action* functions (so they can actually receive more than just two arguments (*element* and *doc*))

**toJSONFilter**(\*args, \*\*kwargs)

Wrapper for [run\\_filter\(\)](#), which calls [run\\_filters\(\)](#)

`toJSONFilter(action, prepare=None, finalize=None, input_stream=None, output_stream=None, **kwargs)` Receive a Pandoc document from stdin, apply the *action* function to each element, and write it back to stdout.

See also [toJSONFilters\(\)](#)

**toJSONFilters**(\*args, \*\*kwargs)

Wrapper for [run\\_filters\(\)](#)

---

**Note:** The *action* functions have a few rules:

- They are called as `action(element, doc)` so they must accept at least two arguments.
- Additional arguments can be passed through the **\*kwargs** of `toJSONFilter` and `toJSONFilters`.
- They can return either an element, a list, or None.
- If they return None, the document will keep the same element as before (although it might have been modified).
- If they return another element, it will take the place of the received element.

- If they return [] (an empty list), they will be deleted from the document. Note that you can delete a row from a table or an item from a list, but you cannot delete the caption from a table (you can make it empty though).
- If the received element is a block or inline element, they may return a list of elements of the same base class, which will take the place of the received element.

### 3.3.5 “Batteries included” functions

These are functions commonly used when writing more complex filters

|                                                              |                                                                                        |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>stringify(element[, newlines])</code>                  | Return the raw text version of an element (and its children elements).                 |
| <code>convert_text(text[, input_format, ...])</code>         | Convert formatted text (usually markdown) by calling Pandoc internally                 |
| <code>yaml_filter(element, doc[, tag, function, ...])</code> | Convenience function for parsing code blocks with YAML options                         |
| <code>debug(*args, **kwargs)</code>                          | Same as print, but prints to <code>stderr</code> (which is not intercepted by Pandoc). |
| <code>shell(args[, wait, msg])</code>                        | Execute the external command and get its exitcode, stdout and stderr.                  |

See also `Doc.get_metadata` and `Element.replace_keyword`

Useful (but not essential) functions for writing panflute filters

#### **class PandocVersion**

Get runtime Pandoc version

use `PandocVersion().version` for comparing versions

**convert\_text**(*text*, *input\_format*='markdown', *output\_format*='panflute', *standalone*=False, *extra\_args*=None, *pandoc\_path*=None)

Convert formatted text (usually markdown) by calling Pandoc internally

The default output format ('panflute') will return a tree of Pandoc elements. When combined with 'standalone=True', the tree root will be a 'Doc' element.

Example:

```
>>> from panflute import *
>>> md = 'Some *markdown* **text** ~xyz~'
>>> tex = r'Some $x^y$ or $x_n = \sqrt{a + b}$ \textit{a}'
>>> convert_text(md)
[Para(Str(Some) Space Emph(Str(markdown)) Space Strong(Str(text)) Space
↳ Subscript(Str(xyz)))]
>>> convert_text(tex)
[Para(Str(Some) Space Math(x^y; format='InlineMath') Space Str(or) Space Math(x_n =
↳ \sqrt{a + b}; format='InlineMath') Space RawInline(\textit{a}; format='tex'))]
```

#### **Parameters**

- **text** (`str` | `Element` | `list` of `Element`) – text that will be converted
- **input\_format** – format of the text (default 'markdown'). Any Pandoc input format is valid, plus 'panflute' (a tree of Pandoc elements)

- **output\_format** – format of the output (default is ‘panflute’ which creates the tree of Pandoc elements). Non-binary Pandoc formats are allowed (e.g. markdown, latex is allowed, but docx and pdf are not).
- **standalone** (*bool*) – whether the results will be a standalone document or not.
- **extra\_args** (*list*) – extra arguments passed to Pandoc
- **pandoc\_path** (*str*) – If specified, use the Pandoc at this path. If None, default to that from PATH.

**Return type** *list* | *Doc* | *str*

Note: for a more general solution, see [pyandoc](#) by Kenneth Reitz.

**get\_option**(*options=None, local\_tag=None, doc=None, doc\_tag=None, default=None, error\_on\_none=True*)

Fetch an option variable from either a local (element) level option/attribute tag, a document level metadata tag, or a default.

**type options** *dict*

**type local\_tag** *str*

**type doc** *Doc*

**type doc\_tag** *str*

**type default** *any*

**type error\_on\_none** *bool*

The order of preference is local > document > default, although if a local or document tag returns None, then the next level down is used. Also, if error\_on\_none=True and the final variable is None, then a ValueError will be raised

In this manner you can set global variables, which can be optionally overridden at a local level. For example, the two files below show how to apply different styles to docx text:

**main.md:**

```

1 -----
2 style-div:
3     name: MyStyle
4 -----
5
6 :::style
7 some text
8 :::
9
10 ::: {.style name=MyOtherStyle}
11 some more text
12 :::

```

**style\_filter.py:**

```

1 import panflute as pf
2
3 def action(elem, doc):
4     if type(elem) == pf.Div:
5         style = pf.get_option(elem.attributes, "name", doc, "style-div.name")

```

(continues on next page)

(continued from previous page)

```

6         elem.attributes["custom-style"] = style
7
8     def main(doc=None):
9         return run_filter(action, doc=doc)
10
11     if __name__ == "__main__":
12         main()

```

**run\_pandoc**(*text=""*, *args=None*, *pandoc\_path=None*)

Low level function that calls Pandoc with (optionally) some input text and/or arguments

**Parameters** **pandoc\_path** (*str*) – If specified, use the Pandoc at this path. If None, default to that from PATH.

**shell**(*args*, *wait=True*, *msg=None*)

Execute the external command and get its exitcode, stdout and stderr.

**stringify**(*element*, *newlines=True*)

Return the raw text version of an element (and its children elements).

Example:

```

>>> from panflute import *
>>> e1 = Emph(Str('Hello'), Space, Str('world!'))
>>> e2 = Strong(Str('Bye!'))
>>> para = Para(e1, Space, e2)
>>> stringify(para)
'Hello world! Bye!

```

**param newlines** add a new line after a paragraph (default True)

**type newlines** *bool*

**rtype** *str*

**yaml\_filter**(*element*, *doc*, *tag=None*, *function=None*, *tags=None*, *strict\_yaml=False*)

Convenience function for parsing code blocks with YAML options

This function is useful to create a filter that applies to code blocks that have specific classes.

It is used as an argument of **run\_filter**, with two additional options: **tag** and **function**.

Using this is equivalent to having filter functions that:

1. Check if the element is a code block
2. Check if the element belongs to a specific class
3. Split the YAML options (at the beginning of the block, by looking for `...` or `---` strings in a separate line
4. Parse the YAML
5. Use the YAML options and (optionally) the data that follows the YAML to return a new or modified element

Instead, you just need to:

1. Call **run\_filter** with **yaml\_filter** as the action function, and with the additional arguments **tag** and **function**

2. Construct a `fenced_action` function that takes four arguments: (`options`, `data`, `element`, `doc`). Note that `options` is a dict and `data` is a raw string. Notice that this is similar to the `action` functions of standard filters, but with *options* and *data* as the new ones.

Note: if you want to apply multiple functions to separate classes, you can use the `tags` argument, which receives a dict of `tag: function` pairs.

Note: use the `strict_yaml=True` option in order to allow for more verbose but flexible YAML metadata: more than one YAML blocks are allowed, but they all must start with `---` (even at the beginning) and end with `---` or `...`. Also, YAML is not the default content when no delimiters are set.

Example:

```
"""
Replace code blocks of class 'foo' with # horizontal rules
"""

import panflute as pf

def fenced_action(options, data, element, doc):
    count = options.get('count', 1)
    div = pf.Div(attributes={'count': str(count)})
    div.content.extend([pf.HorizontalRule] * count)
    return div

if __name__ == '__main__':
    pf.run_filter(pf.yaml_filter, tag='foo', function=fenced_action)
```

## 3.4 Contributing

Feel free to submit push requests. [This guide](#) has some helpful contributing guidelines!

### 3.4.1 License

BSD3 license (following pandocfilter by @jgm)





## INDICES AND TABLES

- `genindex`
- `modindex`
- `search`



## PYTHON MODULE INDEX

### p

- `panflute`, [1](#)
- `panflute.base`, [18](#)
- `panflute.containers`, [18](#)
- `panflute.elements`, [20](#)
- `panflute.io`, [29](#)
- `panflute.table_elements`, [27](#)
- `panflute.tools`, [32](#)



## A

`ancestor()` (*Element method*), 17

## B

`Block` (*class in panflute.base*), 18

`BlockQuote` (*class in panflute.elements*), 20

`BulletList` (*class in panflute.elements*), 20

## C

`Caption` (*class in panflute.table\_elements*), 27

`Citation` (*class in panflute.elements*), 20

`Cite` (*class in panflute.elements*), 21

`Code` (*class in panflute.elements*), 21

`CodeBlock` (*class in panflute.elements*), 21

`container` (*Element attribute*), 18

`content` (*Element attribute*), 17

`content` (*MetaMap property*), 24

`convert_text()` (*in module panflute.tools*), 32

## D

`Definition` (*class in panflute.elements*), 21

`DefinitionItem` (*class in panflute.elements*), 21

`DefinitionList` (*class in panflute.elements*), 22

`DictContainer` (*class in panflute.containers*), 18

`Div` (*class in panflute.elements*), 22

`dump()` (*in module panflute.io*), 29

## E

`Element` (*class in panflute.base*), 16

`Emph` (*class in panflute.elements*), 22

## G

`get_option()` (*in module panflute.tools*), 33

## H

`Header` (*class in panflute.elements*), 22

`HorizontalRule` (*class in panflute.elements*), 23

## I

`Image` (*class in panflute.elements*), 23

`index` (*Element attribute*), 17

`Inline` (*class in panflute.base*), 18

`insert()` (*ListContainer method*), 19

## L

`LineBlock` (*class in panflute.elements*), 23

`LineBreak` (*class in panflute.elements*), 23

`ListItem` (*class in panflute.elements*), 23

`Link` (*class in panflute.elements*), 23

`ListContainer` (*class in panflute.containers*), 18

`ListContainer` (*class in panflute.elements*), 24

`load()` (*in module panflute.io*), 30

`location` (*Element attribute*), 16

## M

`Math` (*class in panflute.elements*), 24

`MetaBlocks` (*class in panflute.elements*), 24

`MetaBool` (*class in panflute.elements*), 24

`MetaInlines` (*class in panflute.elements*), 24

`MetaList` (*class in panflute.elements*), 24

`MetaMap` (*class in panflute.elements*), 24

`MetaString` (*class in panflute.elements*), 24

`MetaValue` (*class in panflute.base*), 18

`module`

`panflute`, 1

`panflute.base`, 18

`panflute.containers`, 18

`panflute.elements`, 20

`panflute.io`, 29

`panflute.table_elements`, 27

`panflute.tools`, 32

## N

`next` (*Element attribute*), 17

`Note` (*class in panflute.elements*), 25

`Null` (*class in panflute.elements*), 25

## O

`offset()` (*Element method*), 17

`OrderedList` (*class in panflute.elements*), 25

## P

`PandocVersion` (*class in panflute.tools*), 32

panflute  
  module, 1  
panflute.base  
  module, 18  
panflute.containers  
  module, 18  
panflute.elements  
  module, 20  
panflute.io  
  module, 29  
panflute.table\_elements  
  module, 27  
panflute.tools  
  module, 32  
Para (*class in panflute.elements*), 25  
parent (*Element attribute*), 16  
Plain (*class in panflute.elements*), 25  
prev (*Element attribute*), 17

## Q

Quoted (*class in panflute.elements*), 25

## R

RawBlock (*class in panflute.elements*), 25  
RawInline (*class in panflute.elements*), 26  
replace\_keyword() (*Element method*), 17  
run\_filter() (*in module panflute.io*), 30  
run\_filters() (*in module panflute.io*), 30  
run\_pandoc() (*in module panflute.tools*), 34

## S

shell() (*in module panflute.tools*), 34  
SmallCaps (*class in panflute.elements*), 26  
SoftBreak (*class in panflute.elements*), 26  
Space (*class in panflute.elements*), 26  
Span (*class in panflute.elements*), 26  
Str (*class in panflute.elements*), 26  
Strikeout (*class in panflute.elements*), 26  
stringify() (*in module panflute.tools*), 34  
Strong (*class in panflute.elements*), 26  
Subscript (*class in panflute.elements*), 27  
Superscript (*class in panflute.elements*), 27

## T

Table (*class in panflute.table\_elements*), 27  
TableBody (*class in panflute.table\_elements*), 28  
TableCell (*class in panflute.table\_elements*), 28  
TableFoot (*class in panflute.table\_elements*), 28  
TableHead (*class in panflute.table\_elements*), 29  
TableRow (*class in panflute.table\_elements*), 29  
toJSONFilter() (*in module panflute.io*), 31  
toJSONFilters() (*in module panflute.io*), 31

## U

Underline (*class in panflute.elements*), 27

## W

walk() (*Element method*), 16

## Y

yaml\_filter() (*in module panflute.tools*), 34